

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический университет
Петра Великого»

Институт компьютерных наук и кибербезопасности
Направление: 02.03.01 Математика и компьютерные науки

Математическая логика и теория формальных языков
ОТЧЁТ О ВЫПОЛНЕНИИ ЛАБОРАТОРНОЙ
РАБОТЫ №2

«Регулярные выражения»

Вариант 31

Студент,
группы 5130201/30102

_____ Михайлова А. А.

Преподаватель

_____ Востров А. В.

«_____» _____ 2026 г.

Санкт-Петербург, 2026

Содержание

Введение	3
1 Математическое описание	4
1.1 Регулярные выражения	4
1.2 Регулярное выражение для данной работы	6
1.3 Конечный автомат	7
1.4 Недетерминированный конечный автомат	7
1.5 Детерминированный конечный автомат	8
2 Особенности реализации	11
2.1 build_automaton	11
2.2 Функция check_fa	12
2.3 generate_valid_string	12
2.4 main	12
3 Результаты работы	14
Заключение	17
Список использованной литературы	18

Введение

Цель работы: реализовать программу, которая проверяет введённый текст через реализацию конечного автомата

Вариант 31: Проверка номера и серии паспорта и загранпаспорта

Задачи работы

1. Построить регулярное выражение для проверки корректности серии и номера паспорта и загранпаспорта;
2. Построить недетерминированный конечный автомат, проверяющий корректность серии и номера паспорта;
3. Детерминировать полученный недетерминированный конечный автомат;
4. Реализовать программу, которая проверяет введенный текст через реализацию конечного автомата;
5. Реализовать функцию случайной генерации верной строки по полученному конечному автомату

1 Математическое описание

1.1 Регулярные выражения

Регулярные выражения — это формальный способ описания регулярных языков над конечным словарём Σ . Они строятся из следующих элементов:

- **Базовые компоненты:**

- $\emptyset$ — пустое множество (не содержит цепочек)
- ε — пустая цепочка
- $a \in \Sigma$ — любой символ из словаря

- **Операции:**

- **Объединение** ($R_1 + R_2$ или $R_1 | R_2$): представляет множество цепочек, принадлежащих R_1 или R_2
- **Конкатенация** ($R_1 R_2$): множество цепочек, образованных соединением цепочки из R_1 и цепочки из R_2
- **Итерация Клини** (R^*): множество цепочек, состоящих из нуля или более повторений цепочек из R
- **Усечённая итерация** (R^+): аналогична итерации, но требует минимум одного повторения ($R^+ = RR^*$)

Приоритет операций:

1. Итерация ($*$) и усечённая итерация ($+$) — наивысший приоритет
2. Конкатенация (без явного символа)
3. Объединение ($+$ или $|$)

Скобки изменяют порядок выполнения операций.

1. Регулярные множества и языки

Регулярное множество — это бесконечное множество цепочек, построенное из символов Σ с использованием операций объединения, конкатенации и итерации. Например:

- Регулярное выражение $ab + ba^*$ задаёт множество:

$$\{ab\} \cup \{b, ba, baa, baaa, \dots\} = \{ab, b, ba, baa, \dots\}$$

- Выражение $(ac)^*b + c^*$ описывает объединение двух множеств:

- Цепочки вида $(ac)^nb$ (например, $b, acb, acacb, \dots$)
- Цепочки из любого числа символов c (включая ε)

Регулярный язык — это подмножество Σ^* , которое может быть описано регулярным выражением. Например, язык всех цепочек из символов a, b, c является регулярным, так как Σ^* задаётся выражением $(a + b + c)^*$.

2. Формальные операции над языками

Для языков $L_1, L_2 \subseteq \Sigma^*$ определены следующие операции:

(а) **Объединение:**

$$L_1 \cup L_2 = \{w \mid w \in L_1 \text{ или } w \in L_2\}$$

Пример: $\{abbc, cb\} \cup \{ba, ccc\} = \{abbc, cb, ba, ccc\}$

(b) **Конкатенация:**

$$L_1 \cdot L_2 = \{w_1w_2 \mid w_1 \in L_1, w_2 \in L_2\}$$

Пример: $\{abbc, cb\} \cdot \{ba, ccc\} = \{abbcba, abbccc, cbba, cbccc\}$

(c) **Итерация Клини:**

$$L^* = \bigcup_{k=0}^{\infty} L^k, \text{ где } L^0 = \{\varepsilon\}, L^{k+1} = L \cdot L^k$$

Пример: $\{abb, c\}^* = \{\varepsilon, abb, c, abbabb, abbc, cc, cabb, \dots\}$

(d) **Усечённая итерация:**

$$L^+ = \bigcup_{k=1}^{\infty} L^k$$

3. Связь регулярных выражений и языков

Регулярные выражения являются формализмом для описания регулярных языков. Каждое выражение соответствует языку, построенному через комбинацию операций:

- Базовые элементы:
 - $\emptyset$ соответствует пустому языку
 - ε — языку, содержащему только пустую цепочку
 - $a \in \Sigma$ — языку $\{a\}$
- Операции расширяют базовые языки:
 - $R_1 + R_2$ задаёт объединение их языков
 - R_1R_2 — конкатенацию

— R^* — итерацию

4. Важность регулярных языков

Регулярные языки играют ключевую роль в теории формальных языков и компиляторов. Они лежат в основе:

- Лексического анализа (распознавание токенов)
- Валидации строк (например, проверка email)
- Поиска шаблонов в тексте

Мощность регулярных языков: Хотя словарь Σ конечен, регулярные языки могут быть бесконечными (например, язык всех цепочек чётной длины). Однако их мощность ограничена — они образуют счётное множество, в отличие от всех возможных языков над Σ , мощность которых континуум.

1.2 Регулярное выражение для данной работы

```
1 r"^(\d{4} \d{6}|\d{2} \d{7})$"
```

1. Внутренний паспорт РФ:

```
1 \d{4} \d{6}
```

Строка должна содержать 4 цифры, затем пробел, затем 6 цифр.

Описание серии паспорта:

```
1 \d{4}
```

Описание номера паспорта:

```
1 \d{6}
```

2. Загранпаспорт РФ:

```
1 \d{2} \d{7}
```

Строка должна содержать 2 цифры, затем пробел, затем 7 цифр.

Описание серии паспорта:

```
1 \d{2}
```

Описание номера паспорта:

```
1 \d{7}
```

1.3 Конечный автомат

Конечный автомат – это формальная модель преобразователей информации с конечной памятью. Выход конечного автомата зависит не только от входа, но и от цепочки входных сигналов, которые были поданы раньше, с момента начала функционирования автомата.

Формально конечный автомат можно определить следующим образом:

$$A = (Q, \Sigma, Y, s_0, \delta, \lambda), \text{ где}$$

Q – конечное множество состояний,
 Σ – конечное множество входных сигналов,
 Y – конечное множество выходных сигналов,
 s_0 – начальное состояние ($s_0 \in Q$),
 $\delta : Q \times \Sigma \rightarrow Q$ – функция переходов,
 $\lambda : Q \times \Sigma \rightarrow Y$ – функция выходов.

Автоматы делят на детерминированные и недетерминированные. Детерминированный автомат отличается от недетерминированного тем, что любой его переход однозначно определяется по текущему состоянию и входному сигналу.

1.4 Недетерминированный конечный автомат

Недетерминированный конечный автомат (НДКА) – математическая модель, описываемая как

$$A = (Q, \Sigma, \delta, s_0, F), \text{ где:}$$

Q – конечное множество состояний,
 Σ – конечный входной алфавит,
 $\delta : Q \times (\Sigma \cup \{\epsilon\}) \rightarrow 2^Q$ – функция переходов,
 s_0 – начальное состояние ($s_0 \in Q$),
 F – множество финальных состояний ($F \subseteq Q$).
В данной работе для НДКА:

- множество состояний $Q_1 = \{s_0, s_1, s_2, s_3, s_4, s_5, s_6, s_7, s_8, s_9, s_{10}, s_{11}, s_{12}\}$
- входной алфавит $\Sigma = \{0...9, ' '\}$
- начальное состояние s_0

На основе регулярного выражения был построен недетерминированный конечный автомат, граф которого приведён на рис. 1.

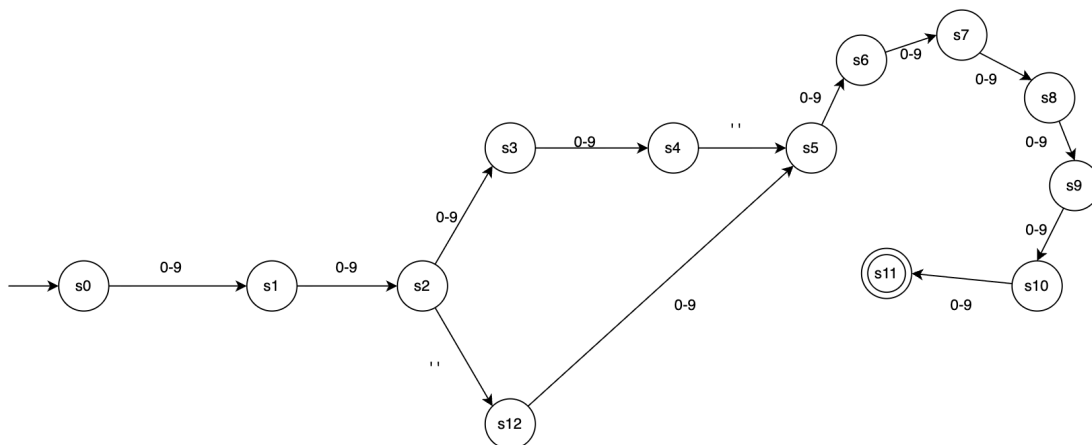


Рис. 1: НДКА

В таблице 1 приведены переходы между состояниями в построенном НДКА.

Таблица 1: Таблица переходов НДКА

Текущее состояние	Входной символ	Следующее состояние
s0	0-9	s1
s1	0-9	s2
s2	0-9	s3
s2	' '	s12
s3	0-9	s4
s4	' '	s5
s5	0-9	s6
s12	0-9	s5
s6	0-9	s7
s7	0-9	s8
s8	0-9	s9
s9	0-9	s10
s10	0-9	s11

1.5 Детерминированный конечный автомат

Детерминированный конечный автомат (ДКА) - математическая модель, описываемая как

$$A = (Q, \Sigma, \delta, s_0, F), \text{ где:}$$

Q – конечное множество состояний,

Σ – входной алфавит,

$\delta : Q \times \Sigma \rightarrow Q$ – функция переходов,
 s_0 – начальное состояние ($s_0 \in Q$),
 F – множество финальных состояний ($F \subseteq Q$).
 В данной работе для ДКА:

- множество состояний $Q_2 = \{Q_1\} \cup \{s_{13}\}$
- входной алфавит $\Sigma = \{0...9, ' '\}$
- начальное состояние s_0

Была выполнена детерминизация имеющегося НДКА, в результате которой был получен детерминированный конечный автомат, граф которого приведён на рис. 2.

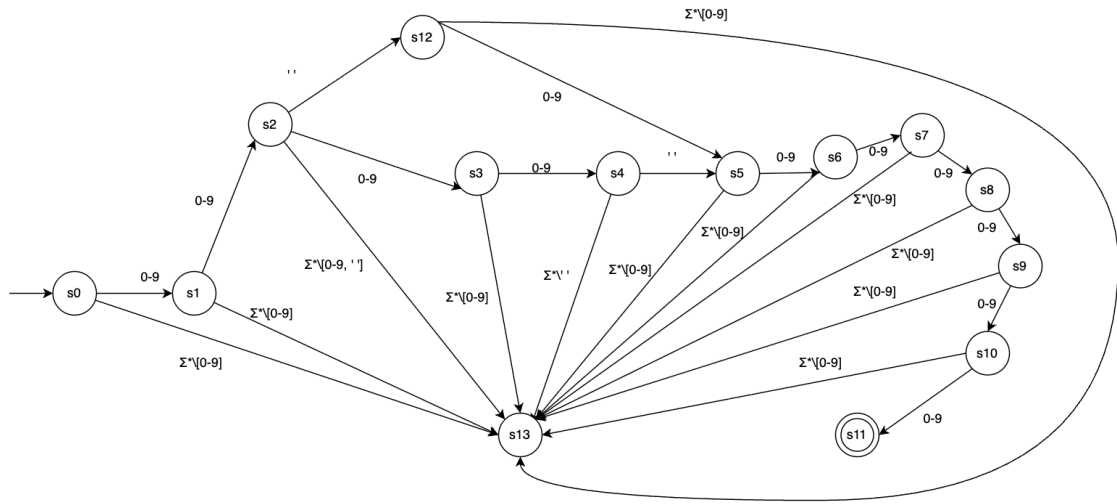


Рис. 2: ДКА

В таблице 2 приведены переходы между состояниями в ДКА.

Таблица 2: Таблица переходов ДКА

Текущее состояние	Входной символ	Следующее состояние
s_0	$0 - 9$	s_1
s_0	$\Sigma^* \setminus [0 - 9]$	s_{13}
s_1	$0 - 9$	s_2
s_1	$\Sigma^* \setminus [0 - 9]$	s_{13}
s_2	$0 - 9$	s_3
s_2	$' '$	s_{12}
s_2	$\Sigma^* \setminus \{0 - 9, ' '\}$	s_{13}
s_3	$0 - 9$	s_4
s_3	$\Sigma^* \setminus [0 - 9]$	s_{13}

Текущее состояние	Входной символ	Следующее состояние
s_4	$' '$	s_5
s_4	$\Sigma^* \setminus ' '$	s_{13}
s_{12}	$0 - 9$	s_5
s_{12}	$\Sigma^* \setminus [0 - 9]$	s_{13}
s_5	$0 - 9$	s_6
s_5	$\Sigma^* \setminus [0 - 9]$	s_{13}
s_6	$0 - 9$	s_7
s_6	$\Sigma^* \setminus [0 - 9]$	s_{13}
s_7	$0 - 9$	s_8
s_7	$\Sigma^* \setminus [0 - 9]$	s_{13}
s_8	$0 - 9$	s_9
s_8	$\Sigma^* \setminus [0 - 9]$	s_{13}
s_9	$0 - 9$	s_{10}
s_9	$\Sigma^* \setminus [0 - 9]$	s_{13}
s_{10}	$0 - 9$	s_{11}
s_{10}	$\Sigma^* \setminus [0 - 9]$	s_{13}

2 Особенности реализации

Множество допустимых символов

```
1 ALPHABET = set("0123456789 ")
```

Класс возможных состояний

```
1 class State(IntEnum):
2     s0 = 0
3     s1 = 1
4     s2 = 2
5     s3 = 3
6     s4 = 4
7     s5 = 5
8     s6 = 6
9     s7 = 7
10    s8 = 8
11    s9 = 9
12    s10 = 10
13    s11 = 11
14    s12 = 12
15    dead_state = -1
```

2.1 build_automaton

Функция отвечает за построение автомата.

Вход: состояния, множество допустимых символов

Выход: заполненная таблица переходов

```
1 def build_automaton():
2     for state in State:
3         if state != State.dead_state:
4             transitions[state] = {}
5
6     for i in range(4):
7         state = State(i)
8         for d in "0123456789":
9             transitions[state][d] = State(i + 1)
10
11    transitions[State.s4][' '] = State.s5
12
13    for i in range(5, 11):
14        state = State(i)
15        for d in "0123456789":
16            transitions[state][d] = State(i + 1)
17
18    transitions[State.s2][' '] = State.s12
19
20    for d in "0123456789":
21        transitions[State.s12][d] = State.s5
```

2.2 Функция check_fa

Функция отвечает за проверку корректности строки.

Вход: входная строка

Выход: результат проверки, выведенный в консоль

```
1 def check_fa(text: str) -> str:
2     state = State.s0
3     for char in text:
4         if char not in ALPHABET:
5             return "символы не из алфавита"
6         next_state = transitions.get(state, {}).get(char, State.
            dead_state)
7         if next_state == State.dead_state:
8             return "не соответствует"
9         state = next_state
10    return "строка соответствует" if state in accept_states else "н
        е соответствует"
```

2.3 generate_valid_string

Функция отвечает за генерацию валидной строки.

Вход: множество допустимых символов

Выход: корректно сгенерированная строка

```
1 def generate_valid_string() -> str:
2     state = State.s0
3     result = []
4     while state not in accept_states:
5         valid_moves = transitions.get(state, {})
6         if not valid_moves:
7             break
8         char = random.choice(list(valid_moves.keys()))
9         result.append(char)
10        state = valid_moves[char]
11    return "".join(result)
```

2.4 main

Функция отвечает за создание интерактивного меню для пользователя.

Вход: инициализация параметров программы

Выход: код завершения программы

```
1 if __name__ == "__main__":
2     print(f"Регулярное выражение: {REGEX}")
3     print("Введите строку для проверки.")
4     print("Введите 'gen' для генерации случайного валидного номера.
        ")
5     print("Введите 'exit' для выхода.")
```

```
6 print("-" * 50)
7 while True:
8     try:
9         user_input = input("> ")
10        except (EOFError, KeyboardInterrupt):
11            print("\nВыход.")
12            break
13
14        cmd = user_input.strip().lower()
15        if cmd in ('exit', 'выход', 'quit'):
16            break
17        if cmd == 'gen':
18            gen_str = generate_valid_string()
19            print(f"\n \"{gen_str}\" -> {check_fa(gen_str)}")
20            continue
21
22        print(check_fa(user_input))
```

3 Результаты работы

При запуске программы мы видим меню – рис. 3.

```
Регулярное выражение: ^(\d{4} \d{6}|\d{2} \d{7})$  
Введите строку для проверки.  
Введите 'gen' для генерации случайного валидного номера.  
Введите 'exit' для выхода.  
-----  
> |
```

Рис. 3: Меню

При вводе gen программа генерирует корректные номер и серию паспорта – рис. 4.

```
> gen  
"7346 866045" -> строка соответствует  
> |
```

Рис. 4: gen

При вводе номера программа проверяет его на корректность и выдаёт ответ – или «строка соответствует», или «не соответствует», или «символы не из алфавита» – рис. 5, рис. 6, рис. 7, рис. 8.

```
> 4010 123456  
строка соответствует
```

Рис. 5: Строка соответствует

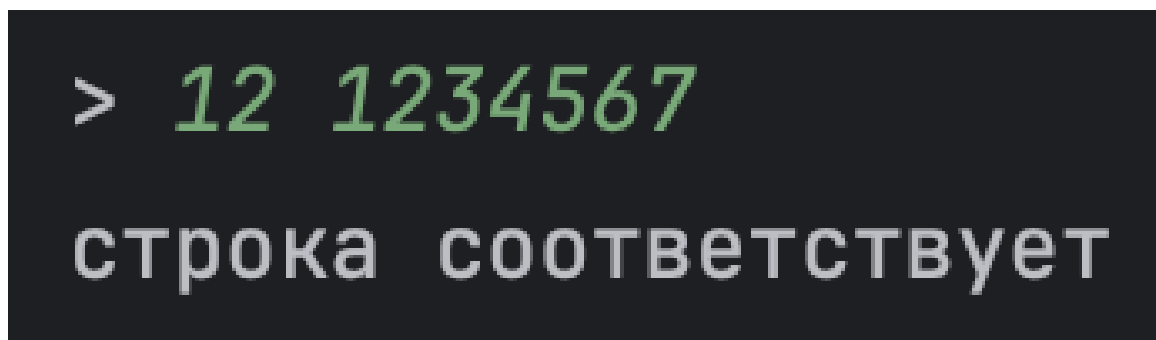


Рис. 6: Строка соответствует



Рис. 7: Не соответствует

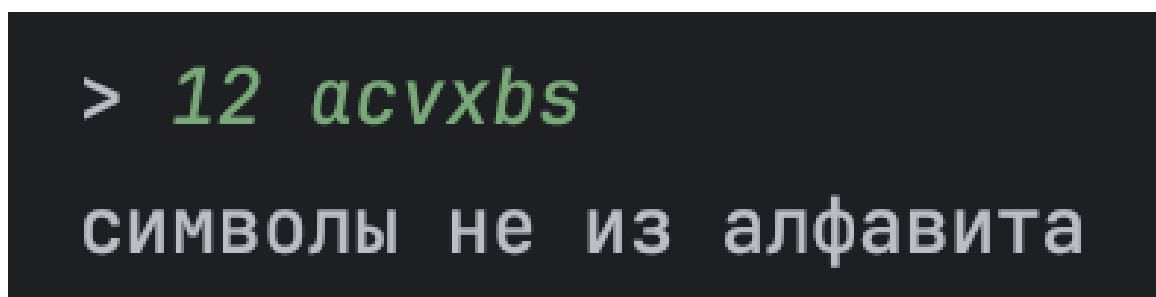


Рис. 8: Символы не из алфавита

При вводе `exit` программа завершается – рис. 9.

```
> exit
```

```
Process finished with exit code 0
```

Рис. 9: exit

Заключение

В результате выполнения лабораторной работы было построено регулярное выражение для проверки корректности серии и номера паспорта и загранпаспорта. Построен и детерминирован недетерминированный конечный автомат. Реализована программа, которая проверяет введенный текст через реализацию конечного автомата. Реализована функция случайной генерации верной строки по полученному конечному автомату.

Достоинства:

- использование Enum для состояний

Недостатки:

- не настроены весовые коэффициенты для равномерной генерации номеров загранпаспорта и внутреннего паспорта;
- программа может сгенерировать несуществующий номер паспорта, например «0000 000000»

Масштабированием программы может быть добавление проверки корректности других данных паспорта: фиио, год рождения, адрес регистрации.

Список литературы

- [1] Секция «Телематика». — Текст: электронный // tema.spbstu: [сайт]. — URL: <https://tema.spbstu.ru/mathem/> (дата обращения: 16.04.2026)